INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
DEPARTMENT OF MECHATRONICS ENGINEERING

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 3: SERIAL INTERFACING WITH MICROCONTROLLER (SENSORS AND ACTUATORS)

SECTION 1

GROUP 10

NO	NAME	MATRIC NUMBER
1.	WIBOWOPUTRI ALYA SHAFIQA	2416728
2.	NURUL SYIFA BINTI RAZALI	2416110
3.	NURUL IZZAH KHALIESA BINTI MOHD NIZAM	2414774
4.	SHARIFAH NUR AZZARIENA EMILY BT SYED MOHD AZAM	2417458

SUBMISSION DATE: 25 OCTOBER 2026

ABSTRACT

The design and implementation of a motion-activated RFID access control system using an Arduino based platform combined with Python processing is the main goal of this experiment. The system includes a servo motor with LEDs for access indication, an RFID reader for user verification, and an MPU6050 accelerometer sensor for motion detection. First, real time accelerometer data is sent via serial communication from the Arduino to Python, where mathematical methods are used to evaluate motion patterns, particularly circular motion. The Arduino receives commands to operate hardware components after successful detection. The second stage introduces RFID technology, which uses unique identifying numbers to confirm permitted users. Lastly, both systems are combined to create a two-factor authentication system that requires the right motion gesture in addition to a valid RFID card in order to allow access. The outcomes emphasize the promise of embedded systems in smart security applications by showing efficient hardware-software communication, precise motion detection, and dependable access control.

TABLE OF CONTENT

TASK 1 : Motion Detection with MPU6050 5
1.0 INTRODUCTION AND THEORY 5
1.1 INTRODUCTION 5
1.2 THEORETICAL BACKGROUND 5
1.2.1 MPU6050 MOTION SENSOR 5
1.2.2 CIRCULAR MOTION DETECTION 6
1.3 EXPERIMENTAL OBJECTIVES 6
1.4 EXPECTED TECHNICAL OUTCOMES 6
2.0 MATERIAL AND EQUIPMENT 7
3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY 7
4.0 METHODOLOGY AND PROGRAM LOGIC 8
4.1 EXPERIMENTAL PROCEDURE 8
4.2 PROGRAM LOGIC AND SYSTEM COMMUNICATION 9
4.3 PROGRAM FLOW 9
4.3.1 ARDUINO FIRMWARE 9
4.3.2 PYTHON HOST SCRIPT 10
4.4 ARDUINO C++ FIRMWARE 10
4.5 PYTHON HOST SCRIPT 12
5.0 DATA COLLECTION AND TECHNICAL ANALYSIS 18
5.1 DATA COLLECTION 18
5.2 TECHNICAL ANALYSIS 19
6.0 RESULTS AND ENGINEERING DISCUSSION 20
6.1 RESULTS 20
6.2 ENGINEERING DISCUSSION 20
7.0 CONCLUSION 21
TASK 2 : Integrated Smart Access System 22
1.0 INTRODUCTION AND THEORY 22
1.1 INTRODUCTION 22
1.2 THEORETICAL BACKGROUND 22
1.2.1 RADIO-FREQUENCY IDENTIFICATION (RFID) 22
1.2.2 MPU6050 MOTION SENSOR 23
1.2.3 SERVO MOTOR AND LED INDICATORS 23
1.2.4 SERIAL COMMUNICATION AND SYSTEM INTEGRATION 23
1.3 EXPERIMENTAL OBJECTIVES 24
1.4 EXPECTED TECHNICAL OUTCOMES 24
2.0 MATERIAL AND EQUIPMENT 25

3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY 25
3.1 HARDWARE CONNECTION SUMMARY 25
3.1.1 ARDUINO UNO AND MPU6050 CONNECTIONS 25
3.1.2 SERVO MOTOR AND LED CONNECTIONS 26
3.1.3 RFID READER AND LAPTOP CONNECTION 26
3.2 HARDWARE ASSEMBLY PROCEDURE 26
4.0 METHODOLOGY AND PROGRAM LOGIC 27
4.1 EXPERIMENTAL PROCEDURE 27
4.2 PROGRAM LOGIC AND SYSTEM COMMUNICATION 28
4.3 PROGRAM FLOW 29
4.3.1 ARDUINO FIRMWARE 29
4.3.2 PYTHON HOST SCRIPT 29
4.4 ARDUINO C++ FIRMWARE 30
4.5 PYTHON HOST SCRIPT 32
5.0 DATA COLLECTION AND TECHNICAL ANALYSIS 35
5.1 DATA COLLECTION 35
5.2 TECHNICAL ANALYSIS 36
6.0 RESULTS AND ENGINEERING DISCUSSION 39
6.1 RESULTS 39
6.2 ENGINEERING DISCUSSION 39
7.0 CONCLUSION 40
FINAL SYNTHESIS SECTION 41
1.0 RECOMMENDATIONS 41
2.0 REFERENCES 41
3.0 APPENDICES 42
4.0 ACKNOWLEDGMENT 43
5.0 STUDENT DECLARATION 43

TASK 1 : Motion Detection with MPU6050

1.0 INTRODUCTION AND THEORY

1.1 INTRODUCTION

Motion tracking is an important application in mechatronics systems because it allows the movement of an object or device to be monitored using sensors. In this experiment, the MPU6050 sensor was interfaced with an Arduino Uno to capture motion data and transmit the measurements to a computer for real-time visualization and analysis.

The MPU6050 is an inertial measurement unit (IMU) that consists of a three-axis accelerometer and a three axis gyroscope. The accelerometer measures acceleration along the X, Y and Z axes, while the gyroscope measures angular velocity around these axes. These measurements provide information about the movement and rotation of the sensor.

In this experiment, the Arduino Uno reads the sensor measurements and transmits the data to a computer through serial communication at a baud rate of 9600 bits per second. A Python program receives the data using the PySerial library and displays the X-axis and Y-axis acceleration values dynamically using Matplotlib. The program also analyzes the collected measurements to identify a circular motion gesture.

The main purpose of this experiment is to understand sensor interfacing, serial communication, real-time data visualization and basic motion detection using Arduino and Python. The experiment also demonstrates how hardware and software can be integrated to monitor and analyze movement.

1.2 THEORETICAL BACKGROUND

1.2.1 MPU6050 MOTION SENSOR

The MPU6050 is a motion sensor that contains an accelerometer and a gyroscope. The accelerometer measures acceleration along the X, Y and Z axes, while the gyroscope measures how quickly the sensor rotates around these axes. These six measurements help the system observe changes in movement.

In this experiment, the Python program mainly uses the X-axis and Y-axis acceleration readings to plot the motion path and detect circular movement. The Z-axis acceleration and the three gyroscope readings are also received and displayed in the Python terminal.

1.2.2 CIRCULAR MOTION DETECTION

Circular motion detection is used to check whether the collected movement data forms a pattern similar to a circle. The Python program analyses the latest 60 X-axis and Y-axis readings to make this decision.

First, the program calculates the centre of the collected points and the distance of each point from that centre. It then checks whether the distances are reasonably similar. Next, it calculates the change in angle between the points and checks whether the movement ends close enough to where it started.

The program identifies circular motion when the total angular change is at least 288°, the relative variation in radius is below 0.50, and the ratio between the starting to ending distance and the average radius is below 0.60. All three conditions must be satisfied. When circular motion is detected, the graph displays “CIRCULAR MOTION DETECTED!” in green. Otherwise, it displays “TRACKING MOTION...” in blue.

These conditions help the program recognise circular like movement. However, the result depends on the quality of the sensor readings and how the sensor is moved during the experiment.

1.3 EXPERIMENTAL OBJECTIVES

The objectives of this experiment are:

1. To connect the MPU6050 sensor to the Arduino Uno and collect motion data.
2. To send accelerometer and gyroscope readings from the Arduino Uno to Python through serial communication.
3. To display the X–Y motion path on a graph in real time using Matplotlib.
4. To develop a Python program that detects circular-like movement using the collected sensor data.
5. To understand how hardware, software and sensor data processing work together in a motion tracking system.

1.4 EXPECTED TECHNICAL OUTCOMES

The expected outcomes were:

- The MPU6050 successfully collects accelerometer and gyroscope readings when movement is detected.
- The Arduino Uno sends the sensor readings to Python through serial communication.
- The Python program displays the X-axis and Y-axis acceleration readings on a graph in real time.
- The circular-motion detection function identifies circular like movement based on the collected data.
- The graph displays the appropriate status to indicate whether circular motion has been detected.

2.0 MATERIAL AND EQUIPMENT

No.	Component	Quantity
1.	Arduino UNO	1
2.	MPU6050 IMU sensor	1
3.	Breadboard	1
4.	Jumper wires	6
5.	USB cables	1

3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY

The experimental setup consisted of an MPU6050 sensor connected to an Arduino Uno, which was connected to a computer through a USB cable. The MPU6050 supplied the motion measurements while the Arduino handled sensor initialisation and data transmission. The Python application running on the computer received the transmitted measurements, displayed the numerical readings and plotted the motion path in real time.

MPU6050 Pin	**Arduino Uno Pin**
VCC	5V
GND	GND
SDA	A4
SCL	A5

Table 3.1: MPU6050 and Arduino Uno Pin Connections

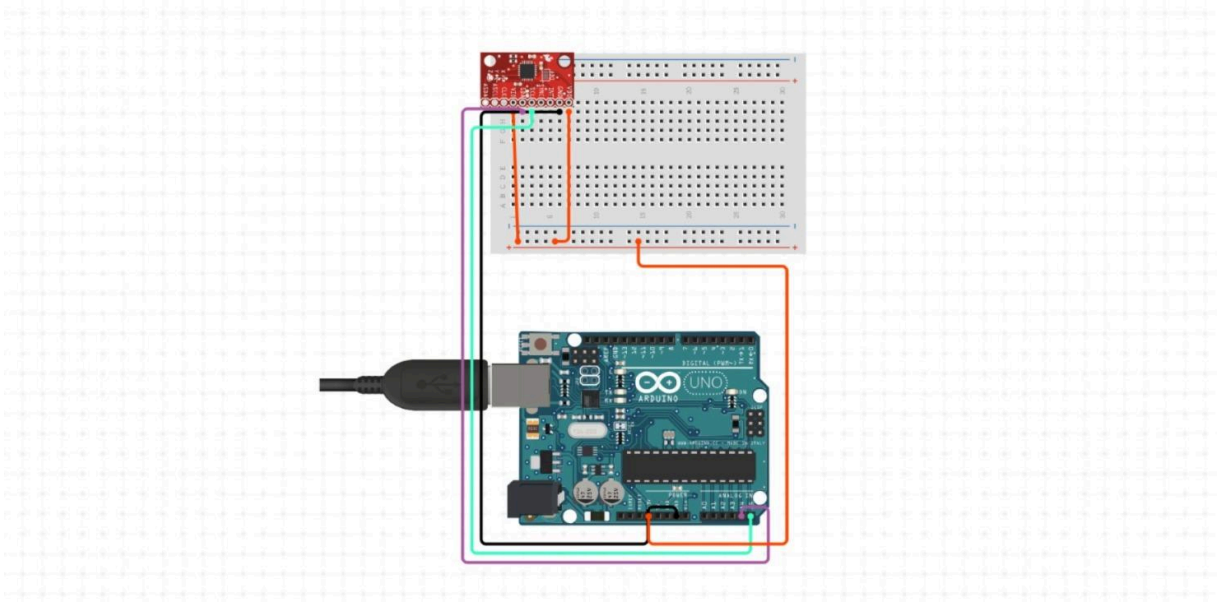


Figure 3.1: Hardware connection between the MPU6050 sensor and Arduino Uno.

4.0 METHODOLOGY AND PROGRAM LOGIC

4.1 EXPERIMENTAL PROCEDURE

The following procedure was carried out to develop and test the real-time motion tracking system using the MPU6050 sensor and Arduino Uno:

1. The MPU6050 sensor was connected to the Arduino Uno using the appropriate circuit connection.
2. The Arduino IDE was opened and the required libraries for the MPU6050 sensor were installed.
3. The Arduino program was written and uploaded to read the accelerometer data along the X and Y axes.
4. The Arduino was connected to a computer via USB cable and transmitted the sensor readings through serial communication.
5. The Python program was executed to receive the accelerometer readings from Arduino and dynamically display the X-Y motion path using Matplotlib.
6. The MPU6050 sensor was moved in circular motion while the graph was monitored to observe the resulting movement pattern and assess the performance of the motion-tracking system.
7. Screenshots and screen recordings of the plotted motion were captured to document the experimental results.

4.2 PROGRAM LOGIC AND SYSTEM COMMUNICATION

- The MPU6050 sensor measures acceleration along the X and Y axes.
- The Arduino reads the sensor data through the serial communication.
- The measured acceleration values are transmitted to the computer through USB serial communication using Serial.println().
- The communication baud rate is set to 9600 bps in both the Arduino and Python programs.
- The Python host script opens the corresponding COM port using the PySerial library.
- Python continuously receives incoming sensor data from the Arduino.
- The received data are decoded from bytes into text and processed into numerical values.
- Matplotlib displays the X-Y motion path for real-time visualization.
- The sensor is moved in a circular motion to observe the resulting pattern on the graph.
- The communication process is one-way, with the Arduino acting as the transmitter and Python acting as the receiver.

4.3 PROGRAM FLOW

4.3.1 ARDUINO FIRMWARE

In this experiment, the Arduino acts as the sensor interface and data transmission unit. It is used to initialize the MPU6050 sensor, collect accelerometer data, and transmit the readings to the Python host script for real-time motion visualization. The Arduino firmware performs the following operations:

1. Initialize serial communication with the computer.
2. Initialize the MPU6050 sensor and establish communication with the sensor.
3. Check whether the MPU6050 sensor is detected successfully.
4. Continuously read accelerometer data along the X and Y axes.
5. Convert the sensor readings into the required data format.
6. Transmit the X and Y acceleration values to the computer through USB serial communication.
7. Repeat the data acquisition and transmission process continuously.
8. Allow the Python host script to receive and process the sensor data for real-time motion tracking.

4.3.2 PYTHON HOST SCRIPT

In this experiment, the Python application acts as the data-processing unit. It is used to establish serial communication with the Arduino, receive MPU6050 sensor data, and visualize the sensor movement in real time. The Python program performs the following operations:

1. Establish serial communication with the Arduino through the appropriate COM port.
2. Continuously receive sensor data transmitted by the Arduino.
3. Decode the received data from bytes into text.
4. Extract the accelerometer readings along the X and Y axes.
5. Convert the received readings into numerical values.
6. Update the X-Y motion plot using the Matplotlib library.
7. Display the sensor movement pattern in real time.
8. Observe the plotted motion to examine the circular movement pattern.
9. Continue updating the graph until the Python program is stopped.

4.4 ARDUINO C++ FIRMWARE

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

Adafruit_MPU6050 mpu;

unsigned long lastRead = 0;
const unsigned long READ_INTERVAL = 100;

bool sensorReady = false;

bool startSensor() {
  if (mpu.begin()) {
    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
    return true;
  }
```

```
  return false;
}

void setup() {
  Serial.begin(9600);

  Wire.begin();
  Wire.setClock(100000);

  // Prevent the I2C bus from waiting indefinitely.
  Wire.setWireTimeout(25000, true);

  Serial.println("Starting MPU6050...");

  sensorReady = startSensor();

  if (sensorReady) {
    Serial.println("MPU6050 READY");
  } else {
    Serial.println("MPU6050 NOT FOUND");
  }
}

void loop() {
  if (millis() - lastRead < READ_INTERVAL) {
    return;
  }
  lastRead = millis();

  if (!sensorReady) {
    Serial.println("Retrying sensor...");
    sensorReady = startSensor();

    if (!sensorReady) {
      return;
    }

    Serial.println("Sensor reconnected");
```

```
  }

  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  // Check whether the I2C peripheral timed out.
  if (Wire.getWireTimeoutFlag()) {
    Wire.clearWireTimeoutFlag();
    Serial.println("I2C TIMEOUT - CHECK SENSOR WIRING");
    sensorReady = false;
    return;
  }

  Serial.print(a.acceleration.x, 3);
  Serial.print(",");
  Serial.print(a.acceleration.y, 3);
  Serial.print(",");
  Serial.print(a.acceleration.z, 3);
  Serial.print(",");
  Serial.print(g.gyro.x, 3);
  Serial.print(",");
  Serial.print(g.gyro.y, 3);
  Serial.print(",");
  Serial.println(g.gyro.z, 3);
}
```

4.5 PYTHON HOST SCRIPT

```
import serial
import matplotlib.pyplot as plt
import numpy as np
from collections import deque
import time

# ====================================
# 1. SERIAL CONNECTION
# ====================================
PORT = "COM6"
BAUD_RATE = 9600
```

```
ser = serial.Serial(PORT, BAUD_RATE, timeout=1)
time.sleep(2)
ser.reset_input_buffer()

print("Arduino connected!")
print("Reading sensor data...")
print("-" * 65)

# ====================================
# 2. DATA STORAGE
# ====================================
MAX_POINTS = 200
WINDOW_SIZE = 60
DETECTION_COOLDOWN = 3

x_vals = deque(maxlen=MAX_POINTS)
y_vals = deque(maxlen=MAX_POINTS)

gesture_x = deque(maxlen=WINDOW_SIZE)
gesture_y = deque(maxlen=WINDOW_SIZE)

last_detection_time = 0
circle_detected = False

# ====================================
# 3. GRAPH SETUP
# ====================================
plt.ion()
fig, ax = plt.subplots(figsize=(9, 7))

line, = ax.plot(
    [], [], "b-", linewidth=1.5,
    label="Motion path"
)

point, = ax.plot(
    [], [], "ro", markersize=6,
```

```
    label="Latest position"
)

ax.set_title("Real-Time X-Y Motion")
ax.set_xlabel("Acceleration X (m/s²)")
ax.set_ylabel("Acceleration Y (m/s²)")
ax.grid(True)
ax.legend()
ax.set_aspect("equal", adjustable="datalim")

# Status stays at the TOP LEFT of the graph
status = ax.text(
    0.02, 0.98,
    "TRACKING MOTION...",
    transform=ax.transAxes,
    va="top",
    ha="left",
    fontsize=11,
    fontweight="bold",
    color="blue",
    bbox=dict(
        facecolor="white",
        edgecolor="gray",
        alpha=0.9
    )
)

# ====================================
# 4. CIRCULAR MOTION DETECTION
# ====================================
def detect_circle(xs, ys):
    if len(xs) < WINDOW_SIZE:
        return False

    x = np.asarray(xs, dtype=float)
    y = np.asarray(ys, dtype=float)

    cx = np.mean(x)
```

```
    cy = np.mean(y)

    dx = x - cx
    dy = y - cy

    radius = np.sqrt(dx**2 + dy**2)
    mean_radius = np.mean(radius)

    if mean_radius < 1e-6:
        return False

    radius_variation = np.std(radius) / mean_radius

    angles = np.unwrap(np.arctan2(dy, dx))
    angle_change = angles[-1] - angles[0]

    start_distance = np.hypot(
        x[-1] - x[0],
        y[-1] - y[0]
    )

    closure_ratio = start_distance / mean_radius

    return (
        abs(angle_change) >= 1.6 * np.pi
        and radius_variation < 0.50
        and closure_ratio < 0.60
    )

# ====================================
# 5. MAIN LOOP
# ====================================
try:
    while plt.fignum_exists(fig.number):

        raw = ser.readline().decode(
            "utf-8", errors="ignore"
        ).strip()
```

```
if not raw:
    plt.pause(0.01)
    continue

values = raw.split(",")

if len(values) != 6:
    continue

try:
    ax_val, ay_val, az_val, gx, gy, gz = [
        float(v) for v in values
    ]
except ValueError:
    continue

readings = [ax_val, ay_val, az_val, gx, gy, gz]

if not np.all(np.isfinite(readings)):
    continue

# ---------------------------------
# PRINT ALL READINGS IN TERMINAL
# ---------------------------------
print(
    f"AX: {ax_val:7.3f} | "
    f"AY: {ay_val:7.3f} | "
    f"AZ: {az_val:7.3f} | "
    f"GX: {gx:7.3f} | "
    f"GY: {gy:7.3f} | "
    f"GZ: {gz:7.3f}"
)

# ---------------------------------
# UPDATE X-Y GRAPH
# ---------------------------------
x_vals.append(ax_val)
```

```
y_vals.append(ay_val)

gesture_x.append(ax_val)
gesture_y.append(ay_val)

line.set_data(list(x_vals), list(y_vals))
point.set_data([ax_val], [ay_val])

ax.relim()
ax.autoscale_view()
ax.set_aspect("equal", adjustable="datalim")

# ---------------------------------
# CIRCULAR MOTION STATUS
# ---------------------------------
detected = detect_circle(gesture_x, gesture_y)
current_time = time.time()

if (
    detected
    and current_time - last_detection_time
    >= DETECTION_COOLDOWN
):
    circle_detected = True
    last_detection_time = current_time

elif not detected:
    circle_detected = False

if circle_detected:
    status.set_text("CIRCULAR MOTION DETECTED!")
    status.set_color("green")
else:
    status.set_text("TRACKING MOTION...")
    status.set_color("blue")

# Update graph only; detection messages stay on graph
fig.canvas.draw_idle()
```

```
        plt.pause(0.001)

except KeyboardInterrupt:
    print("\nProgram stopped by user.")

except serial.SerialException as e:
    print("\nArduino connection error:", e)

finally:
    if ser is not None and ser.is_open:
        ser.close()

    plt.close("all")
    print("Serial connection closed.")
```

5.0 DATA COLLECTION AND TECHNICAL ANALYSIS

5.1 DATA COLLECTION

The data were collected by connecting the MPU6050 sensor to the Arduino and transmitting the accelerometer readings to the computer through serial communication. The Python program received the sensor data and displayed the X-Y motion path in real time using Matplotlib.

The sensor was moved in a circular motion, and the resulting graph was captured to document the experimental results, as shown in Figure 1.

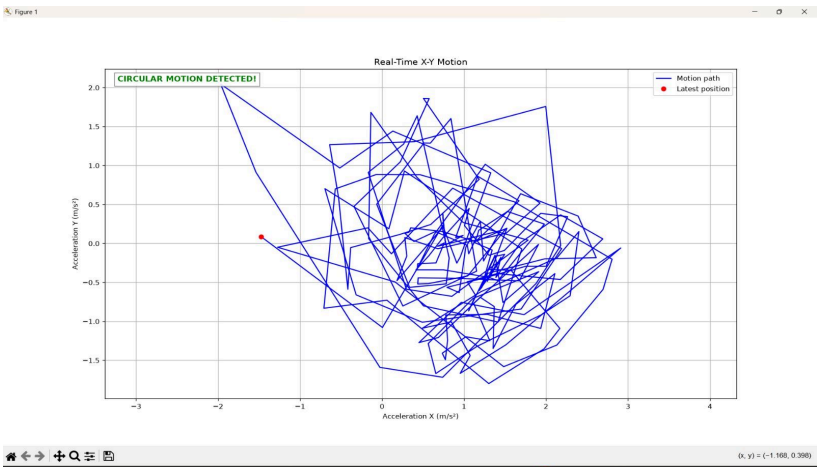


Figure 5.1 Real-Time X-Y Motion Tracking Using the MPU6050 Sensor

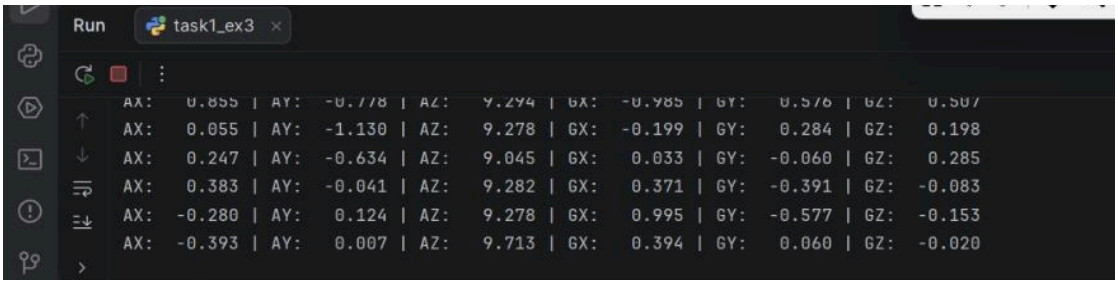


Table 5.1 MPU6050 Motion Sensor Output Collected During Circular Motion

The recorded graph was used to observe the sensor movement pattern and evaluate the real-time motion visualization system.

5.2 TECHNICAL ANALYSIS

The MPU6050 sensor measured acceleration along the X and Y axes as it was moved. The Arduino acquired these readings and transmitted them to the Python program through USB serial communication.

Python processed the received data and plotted the X-Y motion path using Matplotlib. Changes in the acceleration readings caused the plotted points to change continuously, allowing the sensor movement to be visualized in real time.

During circular movement, the X-Y graph was observed to determine whether the plotted pattern corresponded to the movement performed. However, the graph may not form a perfect circle due to sensor noise, changes in orientation, gravitational acceleration, and variations in the speed of movement.

Overall, the experiment demonstrated the integration of sensor data acquisition, serial communication, and real-time graphical visualization using Arduino and Python.

6.0 RESULTS AND ENGINEERING DISCUSSION

6.1 RESULTS

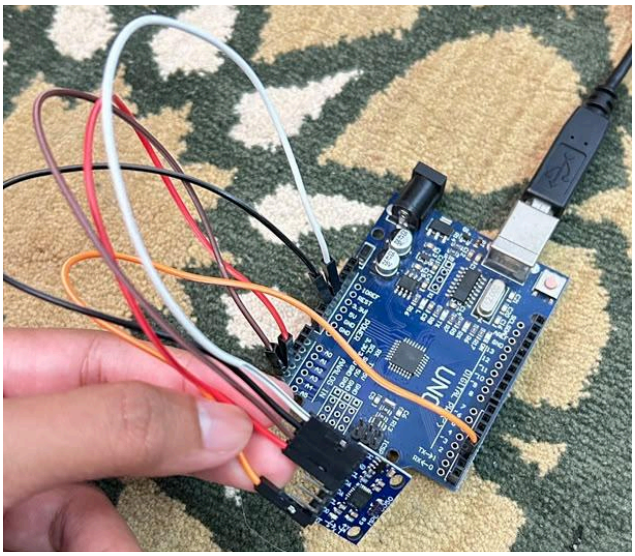


Figure 6.1 Successful Operation of Communication between Arduino Uno and the MPU6050

The experiment was conducted to observe the real-time motion of the MPU6050 sensor using an Arduino and Python-based visualization system. The MPU6050 collected acceleration data along the X and Y axes, which were transmitted to the computer through serial communication.

The Python program received the sensor readings and displayed the data graphically using the Matplotlib library. As the sensor was moved, the plotted points changed according to the incoming sensor data, allowing the movement pattern to be observed in real time.

The resulting graph was used to examine the sensor response during circular movement. The plotted pattern represented changes in the measured acceleration along the X and Y axes during the experiment.

6.2 ENGINEERING DISCUSSION

The experimental graph illustrates how the MPU6050 accelerometer readings varied along the X and Y axes during sensor movement. The irregular and intersecting lines indicate that the acceleration values changed continuously throughout the experiment. This behaviour may be attributed to variations in the direction and speed of movement, sensor orientation, and measurement noise.

One important engineering consideration is that the MPU6050 accelerometer measures acceleration rather than physical position. Therefore, plotting the X-axis acceleration against the Y-axis acceleration does not directly represent the actual

trajectory travelled by the sensor. Even when the sensor is moved in a circular motion, the resulting graph may not form a circle because the plotted coordinates represent acceleration measurements rather than spatial coordinates.

Furthermore, the accelerometer readings may be influenced by gravitational acceleration and changes in the sensor's orientation. These factors can affect the measured values and contribute to variations in the plotted pattern. The graph also shows that the system continuously updates the displayed motion path as new readings are received from the Arduino.

Despite the irregular graph, the experiment demonstrated the functionality of the sensor-data acquisition and real-time visualization system. To improve the accuracy of the motion-tracking results, further development could include sensor calibration, gravity compensation, filtering techniques, and suitable motion-estimation algorithms. If the objective is to reconstruct the actual physical trajectory, additional signal processing would be required to estimate velocity and position from the acceleration data, while accounting for accumulated measurement errors.

In conclusion, the experiment successfully demonstrated real-time communication between the MPU6050 sensor, Arduino, and Python application. However, the results also highlighted the limitations of using raw accelerometer readings alone to represent the actual movement path of an object.

7.0 CONCLUSION

In conclusion, the experiment successfully demonstrated the integration of the MPU6050 sensor, Arduino, and Python for real-time motion-data acquisition and visualization. The MPU6050 measured acceleration along the X and Y axes, while the Arduino transmitted the sensor readings to the computer through serial communication. The Python program then processed the received data and displayed the motion pattern using Matplotlib.

The experimental results showed that the graph updated continuously as the sensor moved, indicating that the system was able to visualize changes in acceleration in real time. However, the plotted motion path was irregular and did not form a perfect circle because the graph represented acceleration measurements rather than the actual physical trajectory of the sensor. Sensor noise, changes in orientation, and gravitational acceleration may also have influenced the results.

Overall, the experiment improved the understanding of sensor interfacing, serial communication, and real-time data visualization in an embedded system. Further improvements, such as sensor calibration, noise filtering, and appropriate motion-estimation techniques, could enhance the accuracy of the system and support more reliable motion tracking in future applications.

TASK 2 : Integrated Smart Access System

1.0 INTRODUCTION AND THEORY

1.1 INTRODUCTION

The Integrated Smart Access System was developed to provide access control by combining radio-frequency identification (RFID), motion sensing, and actuator control. The system uses an RFID reader to identify a user through a card or tag, while an MPU6050 sensor captures hand movements to verify a circular motion gesture. An Arduino Uno, a servo motor, and two LEDs are integrated into the system to control and indicate the access status.

In this implementation, the RFID reader is connected directly to the laptop through USB, allowing the Python program to read the card's unique identification (UID) and check it against a list of authorized cards. The MPU6050 is connected to the Arduino Uno through the I^2C interface, and its motion data is processed by Python to determine whether the required gesture has been performed. Python then communicates the access decision to the Arduino through serial communication. The Arduino controls the servo motor to lock or unlock the system and activates the appropriate LED to indicate whether access is granted or denied.

The experiment aims to demonstrate the integration of identification, motion detection, computer-based processing, and physical actuation in a single access control application.

1.2 THEORETICAL BACKGROUND

1.2.1 RADIO-FREQUENCY IDENTIFICATION (RFID)

RFID is a wireless identification technology that uses radio-frequency communication to exchange information between a reader and an RFID tag or card. Each card has a unique identification number (UID) that can be read by a compatible reader. In this experiment, the RFID reader is connected directly to the laptop through USB, and the Python program retrieves the UID for authentication.

The program compares the detected UID with a predefined list of authorized UIDs. If the UID matches an entry in the list, the user is permitted to proceed to motion verification. Otherwise, access is denied. This authentication process provides the first level of access control. The RFID reader model and its specific communication protocol with the laptop should be confirmed from the hardware used.

1.2.2 MPU6050 MOTION SENSOR

The MPU6050 is an inertial measurement unit (IMU) that integrates a three-axis accelerometer and a three-axis gyroscope. The accelerometer measures linear acceleration along the X, Y, and Z axes, while the gyroscope measures angular velocity around these axes. These measurements can be processed to identify movement patterns.

In this experiment, the MPU6050 is connected to the Arduino Uno using the I^2C communication interface. The sensor's SDA and SCL pins are connected to A4 and A5, respectively. The Arduino reads the motion measurements and transfers the relevant data to the laptop for processing by Python. The program analyzes the collected data to determine whether the user's hand movement corresponds to a circular gesture.

The accelerometer provides information about changes in acceleration, while gyroscope readings can help characterize rotational movement. However, reliable circular-motion detection depends on the actual detection algorithm, sensor orientation, measurement noise, and the criteria used to classify the gesture.

1.2.3 SERVO MOTOR AND LED INDICATORS

A servo motor is an actuator that positions its output shaft according to a commanded angular position. In the experimental system, the servo motor is connected to digital pin D9 of the Arduino Uno. It acts as the physical locking mechanism, moving to the designated unlocked or locked position according to the access decision received from Python.

Two LEDs provide visual feedback. The green LED is connected to digital pin D4, while the red LED is connected to digital pin D3. Each LED uses a 220 Ω series resistor to limit current.

When both RFID authentication and circular-motion verification are successful, the Arduino commands the servo to unlock and turns on the green LED. If the RFID UID is unauthorized or the motion is invalid, the servo remains or returns to the locked position and the red LED turns on. This feedback allows the access status to be identified visually.

1.2.4 SERIAL COMMUNICATION AND SYSTEM INTEGRATION

Serial communication enables the Python program and Arduino Uno to exchange control information through the USB connection between the board and laptop. In the module's example, the communication uses a baud rate of 9600 bits per second. The Python program sends the character A to indicate access approval or D to indicate access denial. The Arduino interprets the received character and updates the servo position and LED states accordingly.

The integrated system uses two separate input paths. The RFID reader sends its data directly to the laptop through USB, while the MPU6050 sends sensor measurements to the Arduino, which forwards the relevant data to Python through serial communication. Python combines the authentication and motion-verification results before sending the final access command to the Arduino.

This arrangement demonstrates how sensor interfacing, computer-side data processing, serial communication, and actuator control can work together in a mechatronic system.

1.3 EXPERIMENTAL OBJECTIVES

The objectives of Task 2 were:

1. To integrate RFID identification and MPU6050-based motion sensing into an access control system.
2. To identify authorized users by reading and verifying RFID UIDs using Python.
3. To analyze motion sensor data to determine whether a circular hand gesture has been performed.
4. To establish serial communication between Python and the Arduino Uno for transmitting access decisions.
5. To control a servo motor and provide green or red LED feedback according to the authentication and motion-verification results.

1.4 EXPECTED TECHNICAL OUTCOMES

The expected outcomes were:

- The RFID reader successfully reads a card UID, and Python determines whether the card is authorized.
- The MPU6050 supplies motion measurements that can be processed to detect the required circular gesture.
- The Python program combines the authentication and motion-verification results to determine whether access should be granted.
- The Arduino receives the access command and controls the servo motor and LEDs accordingly.
- The integrated system grants access only when both the RFID and motion conditions are satisfied.

2.0 MATERIAL AND EQUIPMENT

No.	Component	Quantity
6.	Arduino UNO	1
7.	USB RFID reader	1
8.	RFID card	1
9.	MPU6050 IMU sensor	1
10.	Servo motor	1
11.	Green LED	1
12.	Red LED	1
13.	Current-limiting resistors	2
14.	Breadboard	1
15.	Jumper wires	12 or more
16.	USB cables	2

3.0 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY

3.1 HARDWARE CONNECTION SUMMARY

The system consists of an Arduino Uno, an MPU6050 sensor, a USB RFID reader, a servo motor, and two LED indicators. The MPU6050 and output devices are connected to the Arduino Uno, while the RFID reader is connected directly to the laptop. The laptop runs the Python program that processes the RFID and motion information and communicates the access decision to the Arduino.

3.1.1 ARDUINO UNO AND MPU6050 CONNECTIONS

MPU6050 pin	**Arduino UNO pin**
VCC	5V
GND	GND
SDA	A4

SCL	A5

3.1.2 SERVO MOTOR AND LED CONNECTIONS

Component	**Arduino UNO pin**
Servo signal	D9
Green LED	D4
Red LED	D3
LED resistors	220 Ω in series with each LED

3.1.3 RFID READER AND LAPTOP CONNECTION

Unlike the RFID connection shown in the module, the RFID reader in this experiment is connected directly to the laptop through USB. It does not use the Arduino's D10 and D8 pins for RFID communication.

The Python program reads the UID provided by the USB RFID reader and compares it with the predefined list of authorized cards. If the UID is valid, the program proceeds to motion verification. If the UID is not authorized, the program rejects the access request.

3.2 HARDWARE ASSEMBLY PROCEDURE

The following procedure describes the assembly sequence based on the confirmed setup,

1. Prepare the Arduino Uno. Connect the Arduino Uno to the laptop through USB and ensure that it can be detected by the Arduino IDE.
2. Connect the MPU6050 sensor. Connect VCC to 5V, GND to GND, SDA to A4, and SCL to A5. These connections follow the same configuration used in Task 1.
3. Install the servo motor. Connect the servo signal wire to digital pin D9 and connect its power and ground wires according to the actual circuit arrangement.
4. Assemble the LED indicators. Connect the green LED to D4 and the red LED to D3. Install a 220 Ω resistor in series with each LED, ensuring the correct LED polarity and a suitable ground connection.
5. Connect the RFID reader. Connect the USB RFID reader directly to the laptop using its USB cable. Prepare the required RFID cards or tags and ensure that the reader can obtain their UIDs.

6. Upload the Arduino firmware. Use the Arduino IDE to upload the code for retrieving MPU6050 measurements, communicating with Python, and controlling the servo motor and LED outputs.
7. Prepare the Python program. Configure the program to read RFID data, verify the UID, analyze the MPU6050 motion data, and send the appropriate access command to the Arduino.
8. Test the integrated system. Scan an RFID card, verify its authorization status, and perform the required circular hand gesture. Observe the servo movement and LED indicators to determine whether the system responds according to the access conditions.

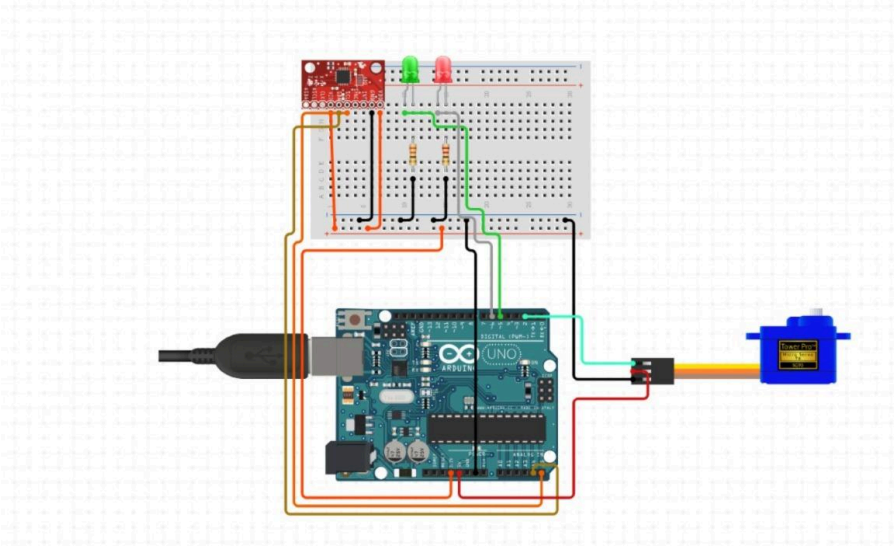


Figure 3.2 shows the circuit schematic design for Task 2

4.0 METHODOLOGY AND PROGRAM LOGIC

4.1 EXPERIMENTAL PROCEDURE

The following procedure was carried out to develop and test the integrated RFID and motion detection system :

1. The Arduino UNO was connected to the computer using a USB cable.
2. The MPU6050 sensor was connected through the interface using SDA, which is connected to A4 and SCL, which is connected to A5.
3. The servo motor, red LED and green LED were connected to their respective digital output pins.
4. The required Arduino libraries, such as Wire.h , SPI.h , MFRC522.h and MPU6050.h were installed.
5. The Python program was executed to establish serial communication with the Arduino through COM6 at 9600 baud.

6. The user entered an RFID card ID into the Python terminal to simulate RFID scanning.
7. The Python program compared the entered RFID ID with the stored authorized ID.
8. If the RFID card was unauthorized, the character 'X' was transmitted to the Arduino to flash the red LED.
9. If the RFID card was authorized, Python instructed the Arduino to stream MPU6050 acceleration data by sending the character 'R'.
10. The Arduino continuously transmitted X-, Y-, Z-axis acceleration data for approximately two seconds.
11. Python collected and analysed the motion data to determine whether a circular gesture had been performed.
12. If circular motion was detected, Python transmitted the character '1' to the Arduino.
13. The Arduino turned on the green LED and rotated the servo from 90° to 180° to unlock the gate.
14. If the motion was invalid, Python transmitted the character '0', causing the servo to return to 90° while the red LED remained ON.
15. The testing procedure was repeated several times using both authorized and unauthorized RFID IDs.

4.2 PROGRAM LOGIC AND SYSTEM COMMUNICATION

- The potentiometer provides an analog input voltage to the Arduino.
- The Arduino reads the analog value from A0 using the analogRead() function.
- The measured value (0–1023) is transmitted through USB serial communication using Serial.println().
- The communication baud rate is set to 9600 bps on both the Arduino and Python programs.
- The Python host script opens the corresponding COM port using the PySerial library.
- Python continuously waits for incoming serial data from the Arduino.
- The received data is decoded from bytes into text.
- The potentiometer value is displayed in the Python terminal for real-time monitoring.
- The communication process is one-way, where Arduino acts as the transmitter and Python acts as the receiver.

4.3 PROGRAM FLOW

4.3.1 ARDUINO FIRMWARE

The Arduino initializes the MPU6050 sensor, servo motor and LED indicators during startup. The red LED is switched ON by default to indicate that the system is locked. The Arduino continuously waits for commands transmitted by the Python application. The received commands are interpreted as follows :

COMMAND	ARDUINO ACTION
R	Stream MPU6050 accelerometer and gyroscope data to Python for approximately 2 seconds.
1	Turn ON green LED and rotate servo from 90° to 180° (open).
0	Turn ON red LED and rotate servo back to 90° (close).
X	Flash the red LED three times to indicate an unauthorized RFID card.

4.3.2 PYTHON HOST SCRIPT

In this experiment, the Python application acts as the decision-making unit. It is used to : establish serial communication with Arduino, verify the scanned RFID card, request MPU6050 sensor data from Arduino, analyse the motion using circular gesture detection and then send the appropriate command back to Arduino. After that, the Python program performs operations as below :

1. Wait for RFID card input.
2. Compare the card ID with the authorized ID.
3. If valid, send ‘R’ to Arduino
4. Collect MPU6050 sensor data
5. Calculate motion angles using the X- and Y-axis acceleration.
6. Remove angle discontinuities using angle unwrapping.
7. Calculate the net rotation and rotation range.
8. Determine whether a circular gesture has been performed.
9. Send '1' for access granted or '0' for access denied.

4.4 ARDUINO C++ FIRMWARE

```
#include <Wire.h>
```

```
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo gateServo;

const int greenLedPin = 4;
const int servoPin = 9;
const int redLedPin = 3;

const unsigned long streamDurationMs = 2000;
const unsigned long sampleDelayMs = 50;
void setup() {
  pinMode(greenLedPin, OUTPUT);
  pinMode(redLedPin, OUTPUT);

  gateServo.attach(servoPin);
  gateServo.write(90);

  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  // Default state: red LED ON (idle/unauthorized)
  digitalWrite(redLedPin, HIGH);
  digitalWrite(greenLedPin, LOW);
}

void openGate() {
  for (int pos = 90; pos <= 180; pos += 2) {
    gateServo.write(pos);
    delay(15);
  }
  Serial.println("Servo opened (90 -> 180)");
}

void closeGate() {
  for (int pos = 180; pos >= 90; pos -= 2) {
    gateServo.write(pos);
```

```
    delay(15);
  }
  Serial.println("Servo closed (180 -> 90)");
}

void wrongCardFlash() {
  digitalWrite(redLedPin, HIGH);
  digitalWrite(greenLedPin, LOW);
  delay(100);

  for (int i = 0; i < 3; i++) {
    digitalWrite(redLedPin, HIGH);
    delay(150);
    digitalWrite(redLedPin, LOW);
    delay(150);
  }
  digitalWrite(redLedPin, HIGH); // Return to idle red ON
}

void loop() {
  if (Serial.available()) {
    char cmd = Serial.read();

    if (cmd == '1') {
      // Access granted
      digitalWrite(greenLedPin, HIGH);
      digitalWrite(redLedPin, LOW);
      openGate();
    }
    else if (cmd == '0') {
      // System disarmed (Lock)
      digitalWrite(greenLedPin, LOW);
      digitalWrite(redLedPin, HIGH);
      closeGate();
    }
    else if (cmd == 'X') {
      // Wrong card feedback
      wrongCardFlash();
    }
    else if (cmd == 'R') {
      // Python requested motion data
      unsigned long start = millis();
      while (millis() - start < streamDurationMs) {
        int16_t ax, ay, az, gx, gy, gz;
```

```
        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

        Serial.print(ax); Serial.print(',');
        Serial.print(ay); Serial.print(',');
        Serial.print(az); Serial.print(',');
        Serial.print(gx); Serial.print(',');
        Serial.print(gy); Serial.print(',');
        Serial.println(gz);

        delay(sampleDelayMs);
      }
    }
  }
}
```

4.5 PYTHON HOST SCRIPT

```
import serial
import time
import math

# --- CONFIGURATION ---
SERIAL_PORT = "COM6"
BAUD_RATE = 9600
AUTHORIZED_ID = ("0013070376")

# ----------------------

def detect_circle(arduino):
    """
    Request MPU data from Arduino and detect circular motion
    """
    print("\n[!] Requesting samples from Arduino… Move in a circle
NOW!")
    arduino.reset_input_buffer()
    arduino.write(b'R')  # trigger Arduino stream

    points = []
    start_time = time.time()
    timeout = 3.0
```

```
    while time.time() - start_time < timeout:
        raw = arduino.readline().decode(errors='ignore').strip()
        if not raw:
            continue

        parts = raw.split(',')
        if len(parts) >= 3:
            try:
                ax = int(parts[0])
                ay = int(parts[1])
                az = int(parts[2])
                points.append((ax, ay, az))
            except ValueError:
                continue

    print(f"Collected {len(points)} samples.")

    if len(points) < 8:
        print("Not enough motion data — try moving more
continuously.")
        return False

    angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]

    unwrapped = [angles[0]]
    for a in angles[1:]:
        prev = unwrapped[-1]
        diff = a - prev

        if diff > 180:
            diff -= 360
        elif diff < -180:
            diff += 360

        unwrapped.append(prev + diff)

    net_rotation = unwrapped[-1] - unwrapped[0]
```

```
    rotation_range = max(unwrapped) - min(unwrapped)

    print(f"Net rotation: {net_rotation:.1f}°, Range:
{rotation_range:.1f}°")

    if abs(net_rotation) > 100 or rotation_range > 140:
        print("-> Circle gesture detected!")
        return True

    deltas = [unwrapped[i+1] - unwrapped[i] for i in
range(len(unwrapped)-1)]
    large_moves = sum(1 for d in deltas if abs(d) > 10)

    if large_moves >= max(6, len(points)//4):
        print("-> Motion accepted as circle.")
        return True

    print("-> Motion insufficient.")
    return False


def main():
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)

    print("System Ready. Waiting for RFID...")

    armed = False

    try:
        while True:
            card = input("\nTap card: ").strip()

            if card == AUTHORIZED_ID:
                armed = not armed

                if armed:
```

```
                    print("Card Accepted – Perform circular
motion!")

                    if detect_circle(arduino):
                        print("UNLOCKED (LED Green)")
                        arduino.write(b'1')
                    else:
                        print("Wrong gesture!")
                        armed = False
                        arduino.write(b'0')

                else:
                    print("LOCKED (LED Red)")
                    arduino.write(b'0')

            else:
                print("WRONG CARD")
                arduino.write(b'X')

    except KeyboardInterrupt:
        print("\nExiting...")

    finally:
        arduino.write(b'0')
        arduino.close()


if __name__ == "__main__":
    main()
```

5.0 DATA COLLECTION AND TECHNICAL ANALYSIS

5.1 DATA COLLECTION

The Arduino transmitted MPU6050 sensor readings to Python after receiving the 'R' command. Each sample consisted of six values representing the accelerometer and gyroscope measurements.

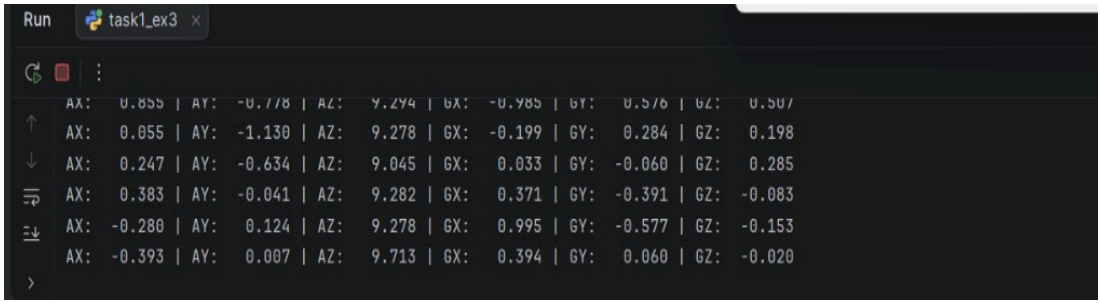


Table 5.1 MPU6050 Motion Sensor Output Collected During Circular Motion

The graph illustrates the relationship between the sample number and the MPU6050 sensor readings, showing the variations in the X-axis (ax), Y-axis (ay), and Z-axis (az) acceleration values during the circular motion performed by the user.

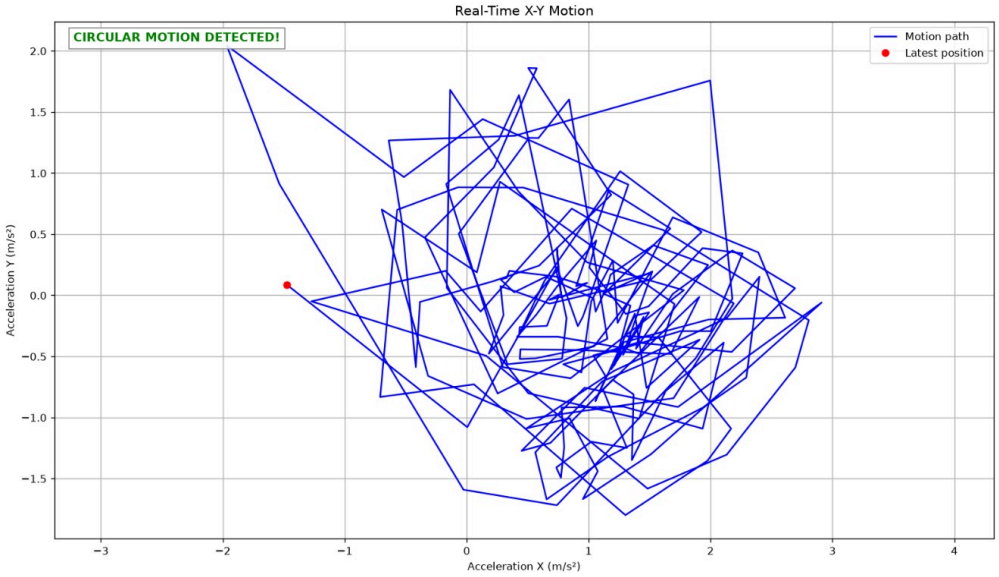


Figure 5.1 MPU6050 Accelerometer Data During Circular Motion Detection

5.2 TECHNICAL ANALYSIS

The Python program analysed the acceleration data received from the MPU6050 sensor to determine whether the user performed the required circular motion gesture.

The motion detection algorithm used the X-axis and Y-axis acceleration values because these two components represent the movement direction on the horizontal plane.

For each collected sensor sample, the movement angle was calculated using the inverse tangent function :

$$\theta = tan^{-1}\left(\frac{a_y}{a_x}\right)$$

Where :

- Θ = calculated motion angle
- a_x = acceleration value along the X-axis
- a_y = acceleration value along the Y-axis

The atan2() function was implemented instead of a normal tangent function because it can determine the correct angle position in all four quadrants. This provides a more accurate representation of the sensor movement direction. During continuous circular motion, the calculated angle may experience sudden discontinuities when the value crosses the boundary between $+180^{\circ}$ and -180°. Hence, an angle unwrapping technique was applied to overcome this issue. The difference between consecutive angle measurements was calculated :

$$\Delta\Theta = \Theta_i - \Theta_{i-1}$$

If the angle exceeded 180°;

$$\Delta\Theta = \Theta_i - 360^{\circ}$$

If the angle was less than 180° ;

$$\Delta\Theta = \Theta_i + 360^{\circ}$$

This process shows a continuous angle trajectory that represents the actual rotational movement of the sensor. After angle correction, the total angular displacement was calculated using the net rotation equation:

$$\Theta_{net} = \Theta_{final} - \Theta_{initial}$$

A larger net rotation value indicates that the sensor has undergone rotational movement. The program also calculated the rotational range to determine the total angular movement covered :

$$rotational\ range\ =\ \Theta_{max}\ -\ \Theta_{min}$$

The circular motion detection was based on predefined threshold values obtained from the processed accelerometer data. The gesture was considered valid when the absolute value of the net rotation exceeded 100°, $|Net\ Rotation| > 100^{\circ}$ or when the rotation range was greater than 140°. The sensor had experienced sufficient rotational movement to be classified as a circular motion based on these threshold values. The software is used to examine the angular changes between successive samples if neither condition was met. If sufficient large angular fluctuations were found to indicate continuous circular motion rather than random hand motion. Nevertheless, the motion was considered acceptable. By lowering false detections, this additional verification strengthened the gesture recognition algorithm's resilience.

After the motion analysis was completed, the Python application sent the relevant command to the Arduino based on the motion detection and the RFID authentication result. The Arduino turned on the green LED and rotated the servo motor to unlock the gate when the program sent the command '1' in response to the presentation of an authorized RFID card and the detection of a valid circular motion. The computer issued the signal '0' to keep the gate locked and the red LED on if the RFID card was approved but the motion did not meet the necessary requirements. When the command 'X' was sent for an unauthorized RFID card, the Arduino displayed the red LED to signal access denial.

6.0 RESULTS AND ENGINEERING DISCUSSION

6.1 RESULTS

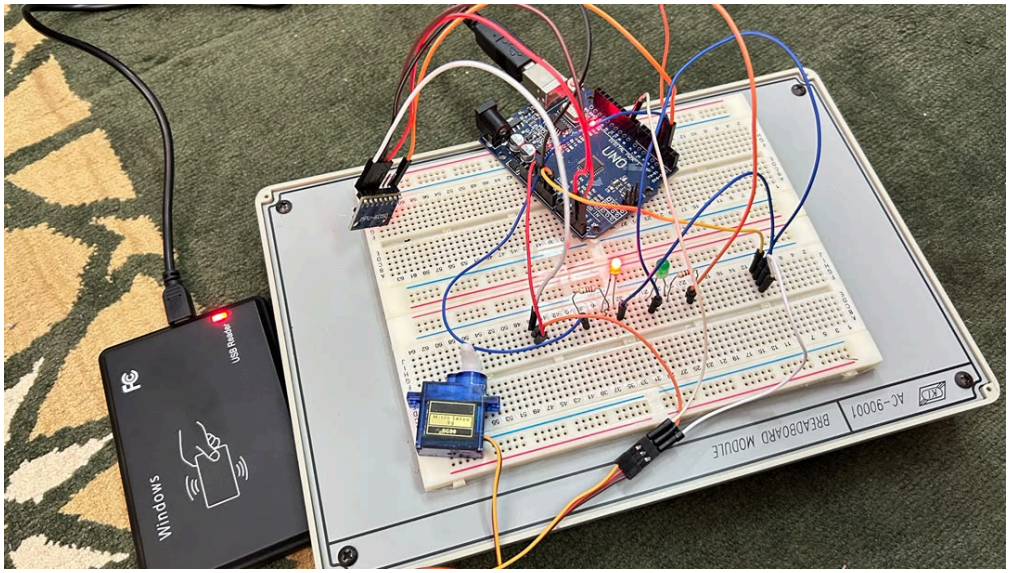


Figure 6.1 Successful Operation of Communication between Arduino Uno and the Python Application

This experiment demonstrated a successful control system implemented by integrating the Arduino Uno, MPU6050, servo motor, LEDs and the Python application through serial communication. The system was able to authenticate the RFID card, analyse the user's hand motion and execute the appropriate access control action based on the verification result.

During testing , the Arduino successfully transmitted MPU6050 accelerometer and gyroscope data to the Python application for motion analysis. The system correctly distinguished between valid and invalid circular motion gestures using predefined threshold values. For a valid RFID card and correct gesture, the green LED turned on and the servo motor rotated from 90° to 180° that will unlock the gate. For invalid gesture, the servo remained locked with the red LED ON while an authorized RFID card caused the red LED to flash three times.

These results confirmed that the integrated authentication and access control system successfully operated.

6.2 ENGINEERING DISCUSSION

The experimental results showed that the security of the access control system was greatly increased by combining RFID identification with MPU6050-based motion recognition. The designed system requires both valid RFID card and proper circular hand

gesture before granting access which is different to the conventional RFID systems that just rely on card authentication. This dual-verification strategy reduced the possibility of unauthorized access if an RFID card was lost or duplicated. By analysing the accelerometer data gathered from the MPU6050 sensor, the circular motion detection algorithm demonstrated success in identifying user motions.

The Python program was able to distinguish between deliberate circular motion and random hand movement by using angle calculation, angle unwrapping, net rotation and rotation range. The threshold-based decision method decreased false detections and increased the accuracy of gesture identification despite slight variations in accelerometer measurements caused by sensor noise and changes in hand movement.

One limitation of this experiment is that the Arduino code was primarily used to verify that the hardware components including the MPU6050 sensor, servo motor and LED indicators were operating as intended rather than performing the complete authentication and motion recognition process independently. The Python program on the computer carried out the majority of the motion analysis and access control logic. As a result, the system's capacity to function as a completely independent embedded access control system is limited because it requires constant serial communication between the Arduino and the PC.

A possible solution to this limitation is to implement the complete authentication and motion recognition algorithm directly on the Arduino or a more powerful microcontroller such as ESP32 instead of relying on the Python application. This would eliminate the need for constant serial communication with a computer and enable autonomous RFID authentication, MPU6050 data processing and servo control. Furthermore, integrating all processing within the microcontroller would enhance the system's performance, reliability and portability. This makes the system suitable for deployment as an independent embedded smart access control system.

7.0 CONCLUSION

This experiment successfully demonstrated and tested a smart access control system by integrating an Arduino Uno, MPU6050 motion sensor, servo motor, LEDs and Python application through serial communication. The system was able to authenticate RFID cards, analyse circular hand gestures using MPU6050 sensor data and control the servo motor based on the verification results. Authorized users with valid circular motions were granted access while unauthorized cards or incorrect gestures were denied access.

All in all, the experiment achieved its objectives by demonstrating reliable hardware-software integration, effective serial communication and accurate motion-based authentication. The combination of RFID authentication and gesture

recognition enhanced the security and reliability of the access control system. This provides a practical application of embedded systems and sensor integration.

FINAL SYNTHESIS SECTION

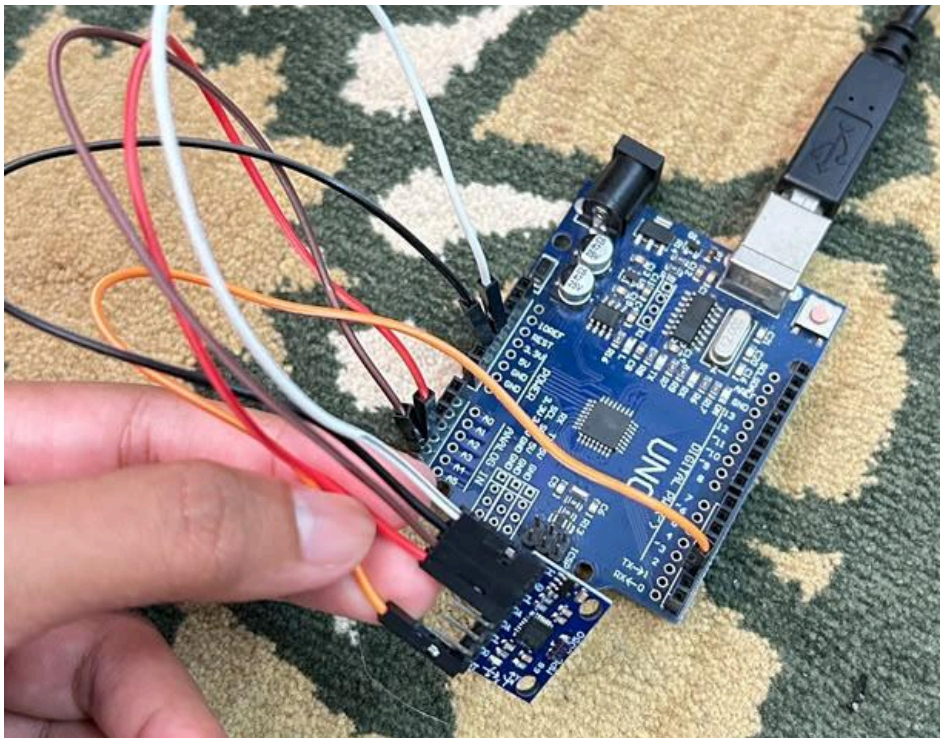
1.0 RECOMMENDATIONS

Few improvements can be made to enhance the reliability and performance of this system. The motion detection algorithm can be improved by applying more advanced techniques or machine learning methods to enhance the accuracy and reduce the false detection. Then, adding a database to store multiple authorized RFID UID as it will be more practical in real life applications where it involves multiple users. The serial communication also should be optimized so it will provide a faster and reliable data transfer. Overall, these improvements would make this system more reliable and suitable for real life world smart access control applications.

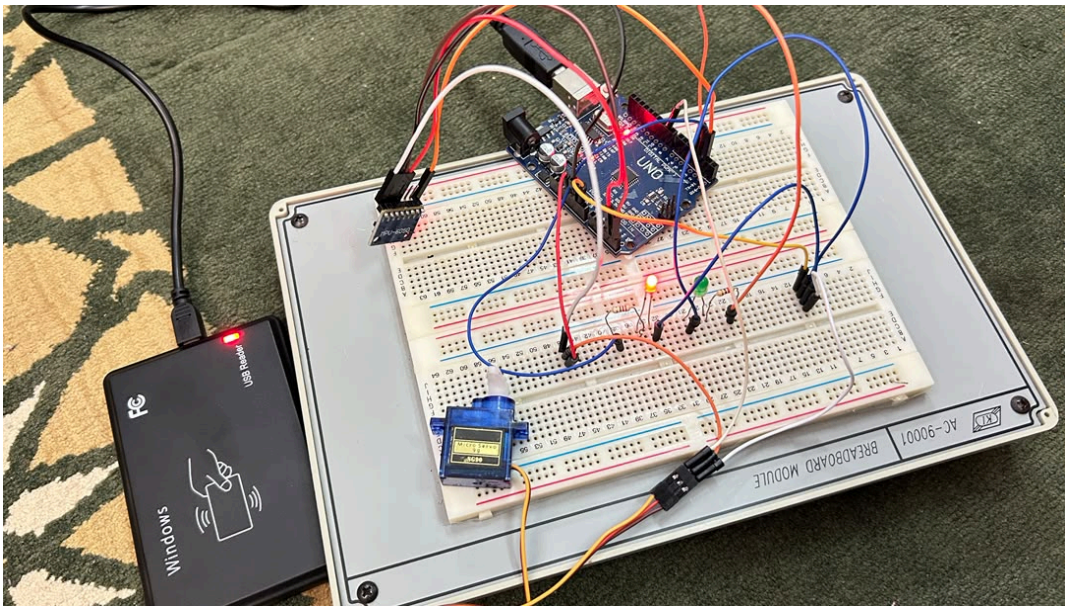
2.0 REFERENCES

Superb Tech. (2021, January 10). *MPU6050 sensor Arduino tutorial* [Video]. YouTube. https://www.youtube.com/watch?v=a37xWuNJsQI

3.0 APPENDICES



Appendix 1: Components setup for Task 1



Appendix 2: Components setup for Task 2

4.0 ACKNOWLEDGMENT

First and foremost, we would like to express my sincere gratitude to Allah SWT for granting us the opportunity, strength and guidance to learn, complete this laboratory work and gain valuable knowledge throughout this experiment. Other than that, we would like to deeply thank Dr. Ahmad Imran Bin Ibrahim for sharing his valuable knowledge and providing continuous guidance throughout the completion of this laboratory work. His assistance and dedication to teaching have helped us gain a better understanding of the concepts covered in this laboratory. Last but not least, we would also like to extend our heartfelt thanks to our fellow friends for their cooperation and support throughout the completion of this laboratory assignment.

5.0 STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: WIBOWOPUTRI ALYA SHAFIQA Matric No: 2416728	Read [/] Understand [/] Agree [/]

	Name: NURUL SYIFA BINTI RAZALI Matric No: 2416110	Read [/] Understand [/] Agree [/]
	Name: NURUL IZZAH KHALIESA BINTI MOHD NIZAM Matric No: 2414774	Read [/] Understand [/] Agree [/]
	Name: SHARIFAH NUR AZZARIENA EMILY BT SYED MOHD AZAM Matric No: 2417458	Read [/] Understand [/] Agree [/]